spotly

Manual de instalación y despliegue

Plataforma web de arriendo temporal de espacios por hora, día o mes

INACAP · Ingeniería en Informática

Proyecto de Integración – TIHI63

Versión 2.0 · octubre de 2026

Repositorio: github.com/Alber-Rios/proyecto (rama main)

Autor: Albert Ríos

Control del documento

Tabla 1
Historial de versiones

Versión	Fecha	Cambios
1.0	12-10-2026	Primera guía de configuración de Supabase y despliegue (archivo DEPLOYMENT_AND_SUPABASE_SETUP.md).
2.0	23-10-2026	Manual completo: requisitos, instalación local, script único de base de datos, variables de entorno, pagos Transbank (Webpay Plus y Oneclick), despliegue en Vercel, alternativas Render y GitHub Pages, verificación, operación y solución de problemas.

Tabla 2
Público al que está dirigido

Perfil	Qué secciones le sirven
Desarrollador que se integra al equipo	Capítulos 2 a 6: dejar el proyecto funcionando en su computador.
Responsable del despliegue	Capítulos 4, 5 y 7 a 9: base de datos, variables, publicación y verificación.
Evaluador o docente	Capítulos 1, 3 y 8: arquitectura, requisitos y cómo comprobar que el sistema funciona.

Contenido

Control del documento 2
1. Introducción 4
1.1 Componentes que se instalan 4
2. Requisitos previos 6
2.1 Hardware 6
2.2 Software 6
2.3 Cuentas en servicios externos 6
3. Obtener el código fuente 7
3.1 Clonar el repositorio 7
3.2 Estructura de carpetas 7
3.3 Instalar las dependencias 7
4. Configurar la base de datos en Supabase 8
4.1 Crear el proyecto 8
4.2 Ejecutar el script único 8
4.3 Configurar la autenticación 9
4.4 Crear el administrador 9
4.5 Seguridad de acceso en Supabase 9
4.6 Envío de correos 10
4.7 Obtener las llaves del proyecto 10
5. Variables de entorno 11
6. Ejecutar Spotly en el computador 12
6.1 Iniciar el servidor de desarrollo 12
6.2 Comandos disponibles 13
6.3 Probar un pago en el ambiente de integración 13
7. Publicar en internet 14
7.1 Opción recomendada: Vercel 14
7.2 Pasar a pagos reales (producción) 14
7.3 Alternativa: Render (un solo servicio) 14
7.4 Demo estática en GitHub Pages 15
8. Verificar la instalación 16
9. Operación y mantenimiento 17
9.1 Actualizar la aplicación 17
9.2 Cambios en la base de datos 17
9.3 Respaldos 17
9.4 Monitoreo 17
10. Solución de problemas 18

1. Introducción

Este manual explica, paso a paso, cómo instalar Spotly en un computador de desarrollo y cómo publicarlo en internet. Spotly es una aplicación web que conecta a personas que necesitan un espacio (oficina, sala de reuniones, estudio, quincho, bodega, etc.) con propietarios que los arriendan por hora, día o mes, con verificación de identidad, contrato digital y pago en línea con Transbank.

El manual parte desde cero: no supone que el lector tenga una base de datos creada ni cuentas configuradas. Al terminar el capítulo 6 el sistema funciona en el computador local; al terminar el capítulo 7 queda publicado en Vercel con una dirección pública.

1.1 Componentes que se instalan

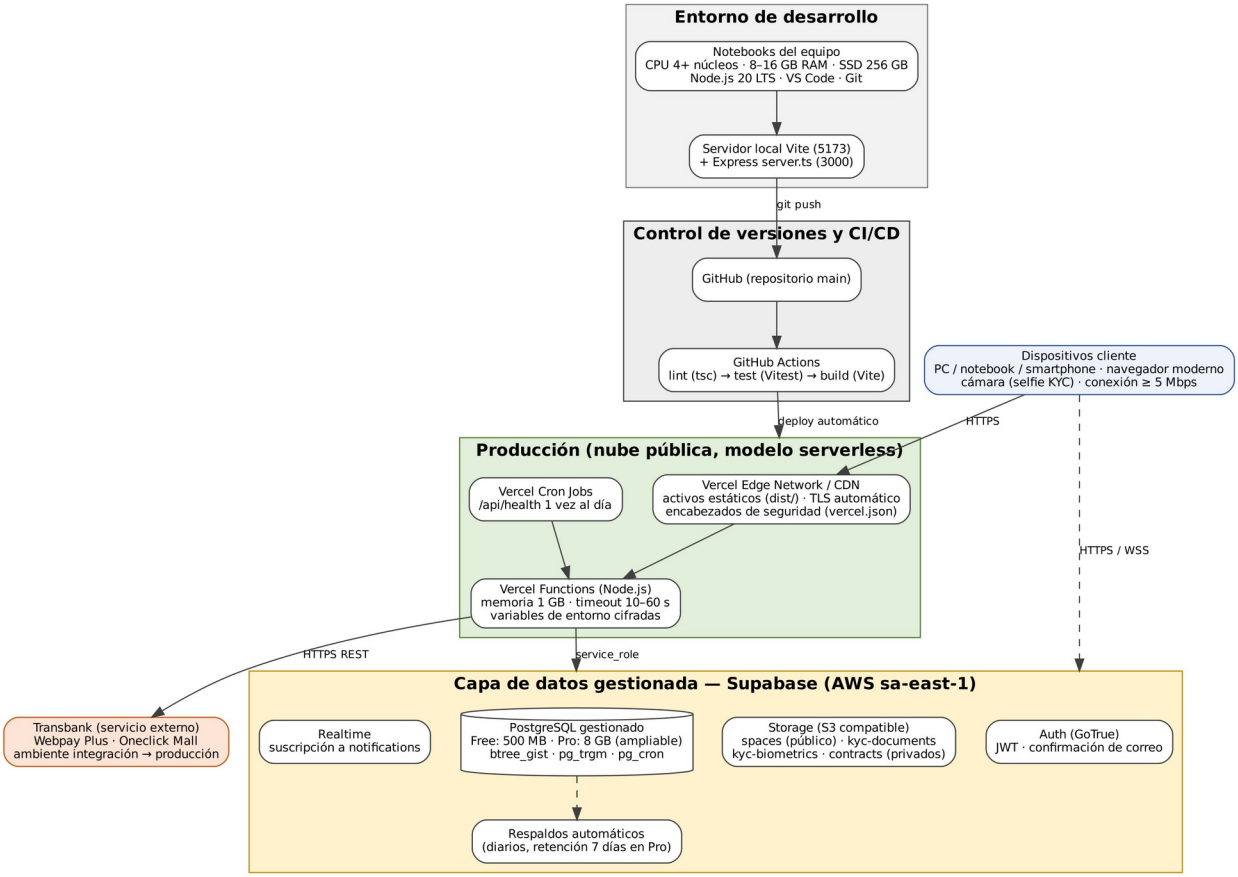
Spotly no necesita servidores propios. Todo se ejecuta en servicios gestionados en la nube:

Tabla 3
Componentes de la solución

Componente	Tecnología	Dónde se ejecuta
Interfaz web (frontend)	React 19, TypeScript 5.8, Vite 6, Tailwind CSS 4, Motion y Anime.js	Navegador del usuario; se publica en Vercel
API propia (backend)	Funciones Node.js en la carpeta `api/` (webpay, wallet, auth, client-ip, health)	Vercel Functions; en local, el servidor `server.ts` (Express)
Base de datos	PostgreSQL con seguridad por filas (RLS), disparadores y tareas pg_cron	Supabase
Autenticación	Supabase Auth (correo y contraseña)	Supabase
Archivos	Supabase Storage: buckets spaces, kyc-documents, kyc-biometrics y contracts	Supabase
Pagos	Transbank Webpay Plus y Oneclick Mall	Transbank (ambiente de integración o producción)

Figura 1

Infraestructura de Spotly: desarrollo, integración continua, producción y datos



Nota. Elaboración propia.

2. Requisitos previos

2.1 Hardware

Tabla 4
Hardware recomendado para el computador de desarrollo

Recurso	Mínimo	Recomendado
Procesador	4 núcleos	6 a 8 núcleos
Memoria RAM	8 GB	16 GB
Disco	2 GB libres (SSD)	5 GB libres (SSD)
Conexión a internet	5 Mbps	20 Mbps o más

2.2 Software

Tabla 5
Software necesario

Programa	Versión	Para qué se usa	Cómo comprobarlo
Node.js (incluye npm)	20 LTS o superior (probado con 20 y 22)	Ejecutar, compilar y probar el proyecto	`node -v` y `npm -v`
Git	2.40 o superior	Descargar el código y subir cambios	`git --version`
Editor de código	Visual Studio Code (recomendado)	Editar el código y las variables	—
Navegador	Chrome, Edge, Firefox o Safari actualizados	Usar y probar la aplicación	—

2.3 Cuentas en servicios externos

Tabla 6
Cuentas necesarias

Servicio	Plan para desarrollo	Obligatorio	Uso
GitHub	Gratis	Sí	Repositorio del código e integración continua
Supabase	Free	Sí	Base de datos, autenticación y archivos
Vercel	Hobby (gratis)	Solo para publicar	Hosting de la web y de la API
Transbank	Ambiente de integración (sin cuenta)	No en desarrollo	Pagos; las credenciales de prueba son públicas

> **Antes de empezar**
> Ten a mano un correo para la cuenta de administrador de Spotly. En el capítulo 4 se le asigna ese rol desde la base de datos.

3. Obtener el código fuente

3.1 Clonar el repositorio

Abre una terminal (en Windows: PowerShell o la terminal de Visual Studio Code) en la carpeta donde quieras guardar el proyecto y ejecuta:

```
git clone https://github.com/Alber-Rios/proyecto.git
cd proyecto
```

Si el repositorio es privado, Git pedirá iniciar sesión en GitHub. Si ya tienes una copia descargada como ZIP, descomprímela y abre la carpeta en la terminal.

3.2 Estructura de carpetas

Tabla 7
Carpetas y archivos principales del repositorio

Ruta	Contenido
`src/pages/`	Pantallas: inicio, detalle del espacio, mis reservas, panel del propietario, panel de administración, billetera, soporte, perfil, verificación, páginas legales.
`src/components/`	Componentes reutilizables: encabezado, logo, modales de reserva y pago, formulario de publicación, gráficos de los paneles, animaciones.
`src/context/AppContext.tsx`	Estado global: sesión, espacios, reservas, billetera y acciones de negocio.
`src/lib/` y `src/utils/`	Acceso a Supabase y reglas de dominio: precios, reembolsos, disponibilidad, contratos (SHA-256), reportes.
`api/`	Funciones del servidor: `webpay.ts`, `wallet.ts`, `auth.ts`, `client-ip.ts`, `health.ts`.
`server.ts`	Servidor Express para desarrollo local (sirve la web y la API en el mismo puerto) y para Render.
`supabase/reset-and-setup.sql`	Script único que crea toda la base de datos (22 tablas, funciones, RLS, Storage, tiempo real y tareas programadas).
`public/`	Archivos estáticos: favicon, especificación OpenAPI (`openapi.yaml`).
`src/**/*.test.ts` y `tests/`	Pruebas automatizadas con Vitest (88 pruebas).
`.env.example`	Plantilla de variables de entorno.
`vercel.json` y `render.yaml`	Configuración de despliegue, cabeceras de seguridad y tarea diaria (cron).
`.github/workflows/deploy-pages.yml`	Integración continua: revisión de tipos, pruebas y publicación de una demo en GitHub Pages.

3.3 Instalar las dependencias

```
npm install
```

El comando descarga las librerías declaradas en `package.json` (React, Supabase, Motion, Anime.js, jsPDF, Express, Vitest, entre otras) en la carpeta `node_modules`. Tarda entre uno y tres minutos la primera vez.

> **Windows: error «la ejecución de scripts está deshabilitada»**
> Si PowerShell bloquea npm, ejecuta una vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` y vuelve a abrir la terminal, o usa la terminal «Command Prompt».

4. Configurar la base de datos en Supabase

4.1 Crear el proyecto

1. Ingresa a **supabase.com** e inicia sesión (puedes usar tu cuenta de GitHub).
2. Presiona **New project**, elige la organización y escribe un nombre, por ejemplo `spotly-chile`.
3. Define una contraseña segura para la base de datos y guárdala en un lugar seguro.
4. En **Region** elige **South America (São Paulo) – sa-east-1**, la más cercana a Chile.
5. Presiona **Create new project** y espera uno o dos minutos a que termine de crearse.

4.2 Ejecutar el script único

Toda la base de datos se crea con un solo archivo: `supabase/reset-and-setup.sql` (unas 3.500 líneas). El script corre en una sola transacción: si algo falla, no se aplica nada.

1. En el panel de Supabase abre **SQL Editor** (ícono `>_` de la barra lateral) y presiona **New query**.
2. Abre `supabase/reset-and-setup.sql` en tu editor, copia **todo** el contenido y pégalo en la consulta.
3. Presiona **Run** (o Ctrl + Enter) y espera el mensaje **Success. No rows returned**.

El script crea:

- **22 tablas**: profiles, spaces, reservations, contracts, payments, payment_refunds, owner_payouts, wallet_cards, reviews, favorites, notifications, kyc_verifications, visit_requests, disputes, support_tickets, user_reports, discount_codes, space_blocks, space_views, login_attempts, user_consents y audit_logs.
- **Seguridad por filas (RLS)** en todas las tablas, permisos mínimos por columna y funciones con `search_path` fijo.
- **Reglas de negocio en el servidor**: restricción anti-sobreventa (no permite dos reservas que se crucen), recálculo de montos, cálculo de reembolsos (`compute_refund`), liberación de reservas no pagadas a los 20 minutos y auditoría de solo adición.
- **Storage**: los buckets `spaces` (público), `kyc-documents`, `kyc-biometrics` y `contracts` (privados) con sus políticas.
- **Tiempo real** (Realtime) y la tarea `spotly-maintenance` de pg_cron que corre cada 5 minutos.

> **Atención: el script es destructivo**
> Borra y vuelve a crear las tablas de Spotly. Conserva las cuentas de `auth.users` y los archivos de Storage, pero se pierden reservas, espacios y pagos. En un proyecto con datos reales, haz un respaldo antes (capítulo 9.3).

> **Si aparece un error con pg_cron**
> Activa la extensión en **Database → Extensions → pg_cron** y vuelve a ejecutar el script. Las extensiones `btree_gist` y `pg_trgm` las activa el propio script.

4.3 Configurar la autenticación

1. Ve a **Authentication → Sign In / Providers → Email** y verifica que **Enable Email provider** esté activo.
2. Deja activo **Confirm email**: Spotly exige verificar el correo con un código antes de entrar (ver 4.5 y 4.6).
3. En **Authentication → URL Configuration** escribe en **Site URL** la dirección de la aplicación (`http://localhost:3000` en local y tu dominio de Vercel en producción) y agrégala también en **Redirect URLs**. Esto hace que los enlaces de confirmación y de recuperación de contraseña vuelvan a Spotly.
4. En **Authentication → Rate Limits** revisa que el límite de inicios de sesión sea suficiente: como el login pasa por el backend, Supabase ve la IP del servidor.

4.4 Crear el administrador

1. Abre la aplicación (capítulo 6) y regístrate con el correo que será administrador.
2. En Supabase → **SQL Editor** ejecuta, reemplazando el correo:

```
UPDATE public.profiles
SET role = 'admin', verification_status = 'verified'
WHERE email = 'tu-correo@ejemplo.cl';
```

3. Cierra sesión y vuelve a entrar. En el menú de usuario aparecerá **Panel de administración**.

> **Cada vez que vuelvas a ejecutar el script**
> Los perfiles se recrean desde `auth.users` con el rol con que cada persona se registró (arrendatario o propietario) y sin verificar. Repite el paso 4.4 para recuperar el administrador y vuelve a aprobar las verificaciones.

4.5 Seguridad de acceso en Supabase

Spotly exige verificar el correo con un código y permite recuperar la contraseña con un enlace de un solo uso. En **Authentication**:

- **Sign In / Providers → Email:** activa **Confirm email**; deja **Email OTP Length** en 6 y **Email OTP Expiration** en 900 segundos.
- **Password requirements:** mínimo 8 caracteres con minúsculas, mayúsculas y dígitos. En plan Pro activa **Prevent use of leaked passwords**.
- **URL Configuration → Redirect URLs:** agrega `https://TU-DOMINIO/restablecer-contrasena` y `https://TU-DOMINIO/verificar-correo`.

Si la base ya tenía datos, ejecuta además el parche no destructivo `supabase/patch-2026-10-27-seguridad-acceso.sql`.

4.6 Envío de correos

Opción A (recomendada): crea una cuenta en Resend, verifica tu dominio y define `RESEND_API_KEY` y `EMAIL_FROM`. Spotly envía sus propios correos con su marca.

Opción B: usa los correos de Supabase. Configura un SMTP propio en **Authentication → Emails → SMTP Settings** (el SMTP incluido solo envía a los miembros del equipo) y edita las plantillas: en **Confirm signup** incluye `{{ .Token }}` (el código) y en **Reset password** usa el enlace `{{ .SiteURL }}/restablecer-contrasena?token_hash={{ .TokenHash }}&type=recovery`. El texto completo de ambas plantillas está en `DEPLOYMENT_AND_SUPABASE_SETUP.md`.

> **Cómo se protege la sesión**
>
> Las contraseñas las guarda Supabase Auth solo como hash bcrypt (irreversible).
>
> La sesión larga vive en la cookie `spotly_rt` (HttpOnly, Secure, SameSite=Strict, Path=/api/auth): JavaScript no puede leerla. El navegador solo guarda en memoria un token de 1 hora.
>
> Por eso la web y `/api` deben estar en el mismo dominio (Vercel lo hace por defecto).

4.7 Obtener las llaves del proyecto

En **Project Settings → API Keys** (o **Data API**) copia los tres valores que pide el capítulo 5:

Tabla 8
Datos de Supabase que se necesitan

Dato en Supabase	Variable de entorno	Dónde se usa
Project URL (por ejemplo, https://abcd.supabase.co)	`VITE_SUPABASE_URL` y `SUPABASE_URL`	Navegador y servidor
Llave anon / publishable	`VITE_SUPABASE_ANON_KEY`	Navegador (protegida por RLS)
Llave service_role / secret	`SUPABASE_SERVICE_ROLE_KEY`	Solo el servidor (pagos, billetera, login)

> **Nunca publiques la llave service_role**
>
> Esa llave salta todas las reglas de seguridad. No la escribas con prefijo `VITE_`, no la subas a GitHub y no la pegues en el navegador.

5. Variables de entorno

Las variables guardan las direcciones y llaves de cada servicio. En local se escriben en un archivo `.env` en la raíz del proyecto; en Vercel se cargan desde el panel (capítulo 7). Copia la plantilla:

```
# macOS / Linux / Git Bash
cp .env.example .env

# Windows (PowerShell)
Copy-Item .env.example .env
```

Luego abre `.env` y completa los valores. Un archivo mínimo para desarrollo queda así:

```
VITE_SUPABASE_URL=https://abcd.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOi...   # llave anon
SUPABASE_URL=https://abcd.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOi...   # solo servidor
WEBPAY_ENV=integration
PORT=3000
NODE_ENV=development
```

Tabla 9
Variables de entorno de Spotly

Variable	Obligatoria	Descripción
`VITE_SUPABASE_URL`	Sí	URL del proyecto de Supabase. Sin ella la app muestra «La plataforma no está conectada a la base de datos».
`VITE_SUPABASE_ANON_KEY`	Sí	Llave pública (anon). También la usa el login del backend.
`SUPABASE_URL`	Recomendada	URL de Supabase para el servidor. Si falta, se usa `VITE_SUPABASE_URL`.
`SUPABASE_SERVICE_ROLE_KEY`	Sí, para pagar	Llave privada del servidor. Sin ella `/api/webpay` y `/api/wallet` responden 503 y no se puede pagar.
`WEBPAY_ENV`	No	`integration` (por defecto, pagos de prueba) o `production`.
`WEBPAY_COMMERCE_CODE` / `WEBPAY_API_KEY`	Solo producción	Código de comercio y llave secreta de Webpay Plus entregados por Transbank.
`ONECLICK_COMMERCE_CODE`, `ONECLICK_CHILD_COMMERCE_CODE`, `ONECLICK_API_KEY`	Solo producción	Credenciales de la billetera (Oneclick Mall): comercio mall, tienda hija y llave secreta.
`RESEND_API_KEY` / `EMAIL_FROM`	Recomendada	Envío de los correos de Spotly (código de verificación y enlace de recuperación) con Resend. Sin ella, los envía Supabase con sus plantillas (capítulo 4.6).
`PUBLIC_APP_URL`	No	URL pública de la app para armar la dirección de retorno de Transbank. Por defecto se usa el dominio de la solicitud.
`SITE_URL`	No	Dominio usado para generar `sitemap.xml` y `robots.txt` al compilar.
`PORT` / `NODE_ENV`	No	Puerto del servidor local (3000) y modo (`development` o `production`).

Variable	Obligatoria	Descripción
`VITE_API_URL` / `CORS_ORIGIN`	Solo si la API va separada	URL del backend en Render y dominio permitido por CORS (capítulo 7.3).

Nota. La app también acepta los nombres `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY` y `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`, que crea automáticamente la integración de Supabase en Vercel.

> **El archivo .env no se sube a GitHub**
> Está listado en `.gitignore`. Cada integrante debe crear el suyo.

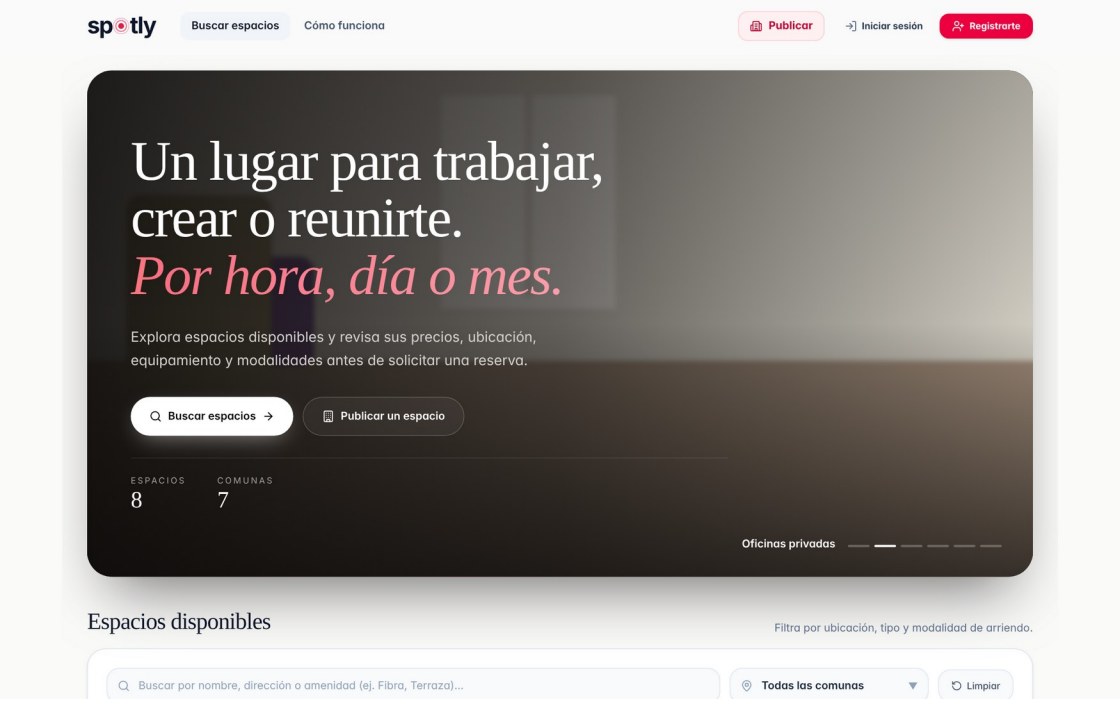
6. Ejecutar Spotly en el computador

6.1 Iniciar el servidor de desarrollo

```
npm run dev
```

El comando ejecuta `server.ts`, que levanta Express con Vite en modo desarrollo: la web y la API quedan en el mismo puerto. Cuando la terminal muestre la dirección, abre **http://localhost:3000** en el navegador. Los cambios en el código se ven al instante, sin recargar.

Figura 2
Página de inicio de Spotly funcionando en local



Nota. Captura del sistema con datos de demostración.

6.2 Comandos disponibles

Tabla 10
Scripts de npm del proyecto

Comando	Qué hace
`npm run dev`	Servidor de desarrollo (web + API) en http://localhost:3000.
`npm run lint`	Revisa los tipos de TypeScript (`tsc --noEmit`). Debe terminar sin errores.
`npm test`	Ejecuta las 88 pruebas automatizadas con Vitest (precios, reembolsos, disponibilidad, contratos, reportes, cancelaciones, billetera).
`npm run test:coverage`	Pruebas con reporte de cobertura de la lógica crítica.
`npm run build`	Compila la versión de producción en la carpeta `dist/`.
`npm run preview`	Sirve la carpeta `dist/` para revisarla antes de publicar (solo la web, sin API).
`npm start`	Ejecuta `server.ts` en modo producción (web compilada + API). Requiere `NODE_ENV=production` y haber corrido `npm run build`.

6.3 Probar un pago en el ambiente de integración

En desarrollo los pagos usan el ambiente de integración de Transbank: no se cobra dinero real. Usa estos datos de prueba:

Tabla 11
Datos de prueba de Transbank

Dato	Valor
Tarjeta de crédito VISA (aprobada)	4051 8856 0044 6623
CVV	123
Fecha de expiración	Cualquier fecha futura
RUT en la autenticación del banco	11.111.111-1
Clave	123

Para probar un pago con tarjeta guardada, inscribe primero la tarjeta en **Billetera** con estos mismos datos.

7. Publicar en internet

7.1 Opción recomendada: Vercel

Vercel compila la web con Vite y publica automáticamente las funciones de la carpeta `api/`. Cada vez que se sube un cambio a la rama `main`, se genera un nuevo despliegue.

1. Sube el código a GitHub (`git push origin main`).
2. Ingresa a **vercel.com**, presiona **Add New… → Project** e importa el repositorio `proyecto`.
3. Verifica la configuración: **Framework Preset**: Vite · **Build Command**: `npm run build` · **Output Directory**: `dist`.
4. En **Environment Variables** agrega las variables del capítulo 5 (como mínimo `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY` y `WEBPAY_ENV`).
5. Presiona **Deploy** y espera a que termine (uno a dos minutos).
6. Copia la dirección pública (por ejemplo, `https://proyecto.vercel.app`) y agrégala en Supabase en **Site URL** y **Redirect URLs** (paso 4.3).

El archivo `vercel.json` ya incluye:

- Reescrituras para que todas las rutas de la web (por ejemplo `/owner` o `/my-bookings`) carguen la aplicación y las rutas `/api/*` vayan a las funciones.
- Cabeceras de seguridad: HSTS, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy` y `Permissions-Policy`.
- Una tarea programada diaria (Vercel Cron) que llama a `/api/health`. Consulta la base de datos y evita que el plan gratuito de Supabase se pause por inactividad.

> **Cambiar una variable no actualiza el sitio publicado**
> Después de modificar variables en Vercel, entra a **Deployments**, abre el último despliegue y presiona **Redeploy**.

7.2 Pasar a pagos reales (producción)

- Firmar la afiliación comercial con Transbank para Webpay Plus y Oneclick Mall.
- Cargar en Vercel `WEBPAY_ENV=production`, `WEBPAY_COMMERCE_CODE`, `WEBPAY_API_KEY` y las tres variables `ONECLICK_*`.
- Usar un dominio propio con HTTPS (por ejemplo, un dominio .cl de NIC Chile) y definir `PUBLIC_APP_URL`.
- Cambiar Supabase y Vercel a planes Pro (sin pausas, con respaldos diarios y uso comercial permitido).

7.3 Alternativa: Render (un solo servicio)

El archivo `render.yaml` permite publicar la web y la API juntas en un servicio Node de Render:

- En Render: **New → Blueprint** y selecciona el repositorio; Render lee `render.yaml`.

- Comando de compilación: `npm install && npm run build`. Comando de inicio: `npx tsx server.ts` con `NODE_ENV=production`.
- Completa las variables marcadas como `sync: false` (llaves de Supabase).

Si se prefiere la web en Vercel y la API en Render, define en Vercel `VITE_API_URL` con la URL de Render y en Render `CORS_ORIGIN` con el dominio de Vercel.

7.4 Demo estática en GitHub Pages

El flujo `.github/workflows/deploy-pages.yml` se ejecuta con cada cambio en `main`: instala dependencias, revisa los tipos, corre las pruebas y publica la web en GitHub Pages bajo la ruta `/spotly/`. Esta demo no tiene funciones de servidor, por lo que no permite pagar; sirve para mostrar la interfaz y como control de calidad automático.

8. Verificar la instalación

Realiza esta lista después de instalar en local y después de cada despliegue:

Tabla 12
Lista de verificación de la instalación

Nº	Prueba	Resultado esperado
1	Abrir `/api/health`	Responde `{"status":"ok","service":"Spotly API"}`.
2	`npm run lint`, `npm test` y `npm run build`	Terminan sin errores; Vitest informa 88 pruebas aprobadas.
3	Abrir la página de inicio	Se ve el logo sp●tly, la portada con fotos y el catálogo (vacío si aún no hay espacios).
4	Registrar un usuario en `/register`	Se crea en Supabase en `auth.users` y en `public.profiles`.
5	Revisar el correo y escribir el código en `/verificar-correo`; luego iniciar sesión	Antes del código la cuenta no puede entrar; después entra y muestra el nombre en el menú. En F12 → Application → Cookies aparece `spotly_rt` con HttpOnly, Secure y SameSite Strict.
6	Enviar la verificación en `/onboarding`	Los archivos quedan en los buckets privados y la solicitud aparece en el panel de administración.
7	Aprobar la verificación como administrador	El usuario queda «Verificado» y puede publicar y reservar.
8	Publicar un espacio y aprobarlo	El espacio aparece en el catálogo.
9	Reservar con la tarjeta de prueba	Aparece la pantalla «Verificando y realizando tu reserva» y luego el resultado; la reserva queda en Mis reservas.
10	Revisar el panel del propietario	La reserva aparece en Solicitudes y Reservas y en los gráficos por día, semana o mes.

9. Operación y mantenimiento

9.1 Actualizar la aplicación

```
git pull origin main
npm install
npm run lint && npm test && npm run build
git push origin main   # Vercel publica automáticamente
```

9.2 Cambios en la base de datos

Toda modificación del esquema se hace en `supabase/reset-and-setup.sql` y se versiona en Git, para que cualquier integrante pueda recrear la base idéntica. No se recomienda crear tablas a mano desde el panel de Supabase, porque esos cambios se pierden al volver a ejecutar el script.

9.3 Respaldos

- **Plan Pro de Supabase:** respaldo diario automático con 7 días de retención (**Database → Backups**).
- **Respaldo manual:** con la cadena de conexión de **Project Settings → Database**:

```
pg_dump "postgresql://postgres:[CLAVE]@db.[ID].supabase.co:5432/postgres" \
  --schema=public --data-only -f respaldo_spotly.sql
```

9.4 Monitoreo

Tabla 13
Dónde revisar el estado del sistema

Qué revisar	Dónde
Errores de las funciones (pagos, billetera, login)	Vercel → proyecto → Logs
Consultas lentas y uso de la base	Supabase → Reports y Query Performance
Tarea de mantenimiento cada 5 minutos	Supabase → Integrations → Cron (`spotly-maintenance`)
Salud general	`/api/health` y la tarea diaria de Vercel Cron
Acciones de usuarios y administradores	Panel de administración → Auditoría

10. Solución de problemas

Tabla 14
Problemas frecuentes y cómo resolverlos

Síntoma	Causa probable	Solución
«La plataforma no está conectada a la base de datos»	Faltan `VITE_SUPABASE_URL` o `VITE_SUPABASE_ANON_KEY`	Complétalas en `.env` (o en Vercel) y reinicia `npm run dev` o vuelve a desplegar.
El pago responde «503» o no avanza	Falta `SUPABASE_SERVICE_ROLE_KEY` en el servidor	Agrégala en `.env` o en Vercel y vuelve a desplegar.
«Could not find the table … in the schema cache»	La API de Supabase no recargó el esquema	Ejecuta `NOTIFY pgrst, 'reload schema';` en el SQL Editor o espera un minuto.
El sitio dejó de responder tras días sin uso	El plan gratuito de Supabase pausó el proyecto	En Supabase presiona Restore project. La tarea diaria de Vercel Cron lo evita.
«Cuenta bloqueada temporalmente» al entrar	5 intentos fallidos con el mismo correo e IP	Espera 15 minutos o usa ¿Olvidaste tu contraseña?.
No llega el código de verificación	Sin `RESEND_API_KEY` ni SMTP propio en Supabase	Configura Resend (4.6, opción A) o un SMTP propio y la plantilla con `{{ .Token }}` (opción B). Revisa spam.
Al iniciar sesión dice que el correo no está verificado	La cuenta no ingresó el código	Presiona Verificar mi correo: se envía un código nuevo.
La sesión se pierde al recargar	La web y la API están en dominios distintos	Publica ambas en el mismo dominio o define `CORS_ORIGIN` con el dominio exacto del frontend.
El enlace de recuperación abre localhost en producción	Site URL de Supabase apunta a local	Actualiza Site URL y Redirect URLs con el dominio de Vercel.
No aparece el Panel de administración	El perfil no tiene `role = 'admin'`	Ejecuta el `UPDATE` del paso 4.4 y vuelve a iniciar sesión.
«Port 3000 is already in use»	Otro programa usa el puerto	Cierra el otro proceso o cambia `PORT` en `.env`.
Error al ejecutar el script SQL	Corrió solo una parte o falta pg_cron	Copia el archivo completo; activa pg_cron (4.2) y vuelve a ejecutarlo.
La reserva desaparece a los 20 minutos	No se completó el pago	Es el comportamiento esperado: libera el horario. Reserva de nuevo y paga.
Las animaciones no se ven	Navegador antiguo o JavaScript bloqueado	Actualiza el navegador. Spotly muestra los efectos aunque el sistema operativo tenga reducidas las animaciones.

Si el problema persiste, revisa la consola del navegador (F12 → Console) y los registros de Vercel, y anota el mensaje exacto del error.